Module 3 — 최적 공정 레시피 추천

> **소유자: P4** — 이 디렉터리는 P4만 수정한다. 상태: 설계 확정 · 구현 전. 아래 결정은 진행하면서 바뀔 수 있으며, 바뀌면 8. 결정 기록에 남긴다. 참고: `MODULE3_OPTIMIZATION_SPEC.md`, `MODULE3_INDEPENDENT_WORK.md`, `PROJECT_PLAN.md`

---

1. 무엇을 하는가

학습된 예측 모델 `f(원료, 공정) → (Ni 회수율, Co 회수율)`을 **가상 공정**으로 삼아, 주어진 원료에 대해 **Ni ≥ 95% · Co ≥ 90%를 달성하는 가장 경제적인 공정 조건**(황산·온도·시간)을 현실적인 제약 안에서 찾고, 작업자가 바로 쓸 수 있는 **레시피**로 출력한다.

새 모델을 학습하지 않는다. P3의 모델을 고정해 두고 **원하는 출력이 나오는 입력을 탐색**한다.

```
원료 성분(고정) + 목표
   │
   ├─ 후보 공정 조건 생성 (optimizers/)
   ├─ feature_fn  ── 지금: 항등 함수   / 나중: P2 add_features
   ├─ predictor   ── 지금: 가짜 모델   / 나중: P3 RecoveryModel   ← config 한 줄로 교체
   ├─ 제약 판정 (constraints.py, domain.py)
   └─ 최적 선택 → 반올림·재검증 → Recipe
                                      │
                                      ▼
                          텍스트 카드 / JSON(→ P5) / CSV
```

---

2. 진행 원칙

- **다른 파트를 기다리지 않는다.** P1·P2·P3 결과물 자리는 인터페이스로 비워두고 가짜 구현으로 먼저 완성한다.
- **다른 모듈 파일은 건드리지 않는다.** 원본 최소 정리는 `analysis/`에서만, 임시 RF는 `artifacts/`(git 제외)에만 둔다.
- **모든 수치는 `config.yaml`에.** 목표치·범위·단위·z값·predictor 종류를 코드에 하드코딩하지 않는다.
- **가짜 모델의 수치는 결과로 쓰지 않는다.** 보고서·발표에 쓸 때는 "개발·검증용 가짜 모델"로 표기한다. 예외: 해석적 모델로 한 **알고리즘 정확성 검증**은 그 자체가 결과다.

---

3. 핵심 설계 결정

3.1 변수

구분	컬럼	비고
결정 변수	`acid_l`(400–1500 L), `temp_c`(40–95 ℃), `time_hr`(1–8 hr)	명세 문장마다 조합이 달라 셋 다 사용

구분	컬럼	비고
주어진 값	`ni_pct`, `co_pct`, `li_pct`, `mn_pct`, `cu_pct`, `fe_pct`, `al_pct`, `grade`, `feed_kg`	`feed_kg`는 배치 크기로 가정 (멘토 확인 필요)
파생 변수	`acid_per_feed`, `impurity_total`, `temp_x_time`, `arrhenius` …	후보마다 **반드시 재계산**

컬럼명은 저장소 뼈대(`module1_preprocessing/cleaning.py`, `features.py`)의 snake_case를 따른다.

3.2 목표 함수

```
기본 (B):   minimize   acid_l
            subject to μ_Ni − z·σ_Ni ≥ 95
                       μ_Co − z·σ_Co ≥ 90
                       u ∈ U  (박스 · 비율 · 현장 단위 · 적용 범위)

해 없음:    ① 목표 미달량 합  max(0, 95−μ_Ni) + max(0, 90−μ_Co)  최소
            ② 같으면 acid_l 최소            → status = "infeasible"
```

- 단순 최대화(A)는 비용을 무시하고 항상 상한으로 몰리므로 채택하지 않는다.
- 달성 불가 시 사전식 비교를 써서 (C)의 임의 가중치 λ를 없앤다. 후보는 적용 범위 안으로 제한한다.
- 비용 단가가 없으므로 황산량을 대리 비용으로 쓴다. `config.yaml`의 `cost.weights`로 확장 가능.

3.3 동점 처리

비용 → 예측 불확실성 → 학습 데이터와의 거리 → 온도 → 시간 (모두 작은 쪽 우선)

3.4 제약과 처리 방식

제약	내용	처리 방식
박스	데이터 사전 정상 범위	탐색 공간 제한
비율	`acid_per_feed`가 관측 1~99 백분위 안	필터 / 페널티
현장 단위	황산 10 L · 온도 1 ℃ · 시간 0.5 hr	격자 자체를 현장 단위로 생성
목표 회수율	Ni ≥ 95, Co ≥ 90	필터 / 페널티
확률 제약	`μ − 1.28σ ≥ 목표` (단측 90%)	필터 / 페널티
적용 범위	표준화 공간 kNN 평균 거리 ≤ 학습 데이터 95 백분위	필터 / 페널티

- 모든 제약 함수는 `(만족 여부: bool Series, 위반량: float Series)`를 반환한다. Grid·Random은 bool, GA·Optuna는 페널티, NSGA-II는 위반량을 `G`로 쓴다.
- 박스는 데이터 사전 범위, 실측 범위 바깥은 적용 범위 필터가 담당한다.

3.5 z값 두 가지를 섞지 않는다

용도	z	의미
확률 제약 판정	**1.28**	단측 90% — P(실제 ≥ 목표) ≥ 0.9
카드·JSON의 예측 구간	**1.645**	양측 90% 구간 — `low`, `high`

σ가 없으면(P3 모델이 std 미제공) σ = 0으로 두고 신뢰도는 최대 `medium`으로 제한한다.

3.6 알고리즘

알고리즘	역할	메모
Grid	기본 엔진 (필수)	현장 단위 격자 111 × 56 × 15 ≈ 9.3만 점, 배치 예측 1회. 반올림 불필요
Grid (조밀도 낮춤)	비교용	50 L · 5 ℃ · 0.5 hr ≈ 4,140점
Random	기준 비교	같은 예산
Optuna TPE	권장 고급 기법	`suggest_float(step=현장단위)`, `constraints_func`
GA (pymoo)	권장 고급 기법	정수 인코딩으로 현장 단위 직접 반영
NSGA-II (pymoo)	보너스	아래 3.7

선택 근거: 트리 모델은 계단 함수라 기울기가 0 → 미분이 필요 없는 방법만 사용. 변수가 3개라 Grid가 사실상 전역 최적을 보장하므로 기본으로 쓰고, 고급 기법은 "더 적은 호출로 같은 해" 및 변수 확장 대비로 평가한다.

3.7 다목적 (보너스)

- Ni–Co 회수율 상관이 **r = 0.915**라 (Ni, Co) 쌍은 파레토가 거의 점 하나가 된다.
- 목표 쌍: **(안전 마진 최대, 황산량 최소)**, 안전 마진 = `min(μ_Ni − 95, μ_Co − 90)`
- r = 0.915는 배치 간 상관이므로, **원료 고정 시 예측 표면에서도 상충이 약한지** 그림으로 확인해 근거로 남긴다.
- 무릎점을 "가성비 추천"으로 제시하고, 마진 = 0 지점이 단일목적 (B) 해와 일치하는지 검증한다.

---

4. 인터페이스 (P4가 가정하는 것)

팀 `docs/INTERFACE.md`가 확정되면 그쪽을 따르고 이 절을 갱신한다.

4.1 받는 것

출처	가정	지금 대체
P3 `RecoveryModel.predict(X) -> ndarray (n, 2)`	여러 행 입력, 열 순서 `[ni, co]` (저장소 뼈대 기준)	`MockAnalyticPredictor`, `ProxyRFPredictor`
P3 불확실성	있으면 사용, 없으면 σ = 0	가짜 모델이 σ 반환
P3 out-of-fold 예측	백테스트용 (없으면 백테스트는 리허설로만)	임시 RF의 OOF
P2 `add_features(df) -> df`	순수 함수, 결정 변수만 바꿔 다시 넣어도 동작	`identity_features`

출처	가정	지금 대체
P1 정제 데이터	`data/processed/`	`analysis/`의 최소 정리본

내부에서는 모든 예측기를 아래 형태로 맞춘다 (`SklearnPairAdapter` 등 어댑터 사용).

```
class Predictor(Protocol):
    def predict(self, X: pd.DataFrame) -> pd.DataFrame:
        """같은 행 수·같은 index. 컬럼: ni_pred, co_pred, ni_std, co_std"""
```

4.2 주는 것 — `recommend()`

```
recommend(feed: dict, target: dict | None = None, algo: str = "grid") -> Recipe
Recipe.to_dict() -> dict      # 아래 JSON
```

- `feed` 필수 키: `ni_pct, co_pct, li_pct, mn_pct, cu_pct, fe_pct, al_pct, feed_kg` / 선택: `batch_id`, `grade`
- 필수 키가 없거나 NaN이면 `InputError`를 던진다 (API에서는 422로 매핑). 원료 결측은 대치하지 않는다.

4.3 출력 JSON

```
{
  "schema_version": "1.0",
  "status": "ok",
  "batch_id": "BP-2025-0412",
  "target": { "ni_recovery_pct": 95.0, "co_recovery_pct": 90.0 },
  "recipe": { "acid_l": 820, "temp_c": 85, "time_hr": 4.5, "acid_per_feed": 5.5 },
  "prediction": {
    "ni": { "mean": 95.8, "low": 94.1, "high": 97.2 },
    "co": { "mean": 91.3, "low": 89.0, "high": 93.1 }
  },
  "shortfall": null,
  "confidence": "medium",
  "similar_batches": 7,
  "domain_distance": 0.84,
  "message": "목표 달성 가능",
  "meta": {
    "algorithm": "grid", "n_evals": 93240, "elapsed_ms": 820,
    "model_version": "mock_analytic", "config_hash": "…", "generated_at": "…"
  }
}
```

`status`	뜻	`recipe` / `prediction`	`shortfall`
`ok`	목표 달성 조건 있음	추천 조건	`null`

status	뜻	recipe / prediction	shortfall
infeasible	적용 범위 안에서 목표 달성 불가	**가장 근접한 조건**	{ "ni": 2.6, "co": 1.9 } (%p)
out_of_domain	원료 자체가 학습 범위 밖	null	null

(수치는 형식 예시다.)

---

5. 디렉터리 구성

```
module3_optimization/
├── README.md              이 문서
├── config.yaml            목표 · 범위 · 현장 단위 · z값 · 적용 범위 · predictor 종류
├── problem.py             Problem (evaluate: 배치 입력), Recipe
├── predictors.py          Predictor Protocol, MockAnalytic, ProxyRF, 어댑터
├── constraints.py         박스 · 비율 · 목표 · 확률 · (만족, 위반량)
├── domain.py              DomainChecker (kNN 거리, 유사 배치 수)
├── optimizers/
│   ├── base.py            Optimizer Protocol, OptResult(history, best, n_evals,
elapsed_s)
│   ├── grid.py
│   ├── random_search.py
│   ├── bayesian.py        Optuna
│   └── genetic.py         pymoo GA
├── multi_objective.py     NSGA-II · 무릎점 (기존 뼈대 파일 사용)
├── recommend.py           원료 → 최적화 → 재검증 → 신뢰도 → Recipe, CLI
├── optimize.py            recommend() 재노출 (기존 뼈대 진입점 유지)
├── report.py              텍스트 카드 · JSON · CSV
├── visualize.py           등고선 · 실행 가능 영역 · 수렴 곡선 · 파레토
├── compare.py             알고리즘 비교 하네스
├── analysis/              원본 직접 분석 (설계 근거용, 정제 산출물 아님)
└── tests/
```

---

6. 로드맵

1단계 — 기반

- [ ] `analysis/`: 원본 최소 정리 (중복 · 등급 · 센티널만, 대치 없음)
- [ ] `acid_per_feed` 분포와 1~99 백분위 → 비율 제약 값
- [ ] 목표 달성 6건 상세표 (등급 · 비율 · 온도 · 시간의 위치)
- [ ] 결정 변수 공간의 빈 영역 확인
- [ ] `config.yaml`, `problem.py`(`Problem`, `Recipe`), `predictors.py`(Protocol)
- [ ] `MockAnalyticPredictor` — 상수를 실제 데이터 수준에 맞춰 **목표 달성 영역이 좁게나마 존재**하도록 보정 (평균 기준 / 하한 기준 두 경우 모두 영역이 사라지지 않는지 확인)
- [ ] `evaluate()`가 1행과 N행에서 같은 결과를 내는지 테스트

2단계 — 필수 산출물 완성

- [ ] `constraints.py` + 경계값 단위 테스트
- [ ] `optimizers/grid.py` (현장 단위 격자, 배치 예측 1회)
- [ ] 정답 검증: 해석 모델에서 (T, t)마다 이분 탐색으로 최소 황산량 → Grid 결과와 일치
- [ ] `recommend.py`: out_of_domain → 최적화 → infeasible 대체 → 신뢰도 → Recipe
- [ ] `report.py`: 텍스트 카드 · JSON · CSV · 달성 불가 안내
- [ ] CLI: `--feed`, `--batch-id`, `--all --out`

> 여기까지가 필수 산출물 3종(목표·제약 설정 / 알고리즘 / 결과 출력)이다.

3단계 — 외삽 방지

- [ ] `ProxyRFPredictor` (결측 없는 행만, 트리별 std, `artifacts/`에만 저장)
- [ ] `domain.py`: 거리 계산 컬럼 비교 (결정 변수만 vs 원료 포함), 백분위 90/95/99별 실행 가능 영역 크기
- [ ] 필터 on/off 비교 그림 — 끄면 추천이 데이터 없는 곳으로 가는지
- [ ] 임시 RF로 동점 수백 개 상황 점검

4단계 — 실제 모델 연결 (P3 모델 도착 시)

- [ ] `config.yaml`의 `predictor.type: real`, 어댑터 연결
- [ ] `feature_fn = add_features`로 교체, 결정 변수만 바꾼 입력으로 재계산 테스트
- [ ] `DomainChecker`를 정제 데이터로 다시 학습
- [ ] 테스트 전체 재실행, 신뢰도 임계값 재조정
- [ ] 7:3 혼합 원료 시나리오 데모

5단계 — 고급 기법 · 보너스

- [ ] `random_search.py`, `bayesian.py`, `genetic.py`
- [ ] `compare.py`: 원료(등급별 대표 + 목표 달성 6건) × 알고리즘 × seed 5개 → 최적 비용 · 정답 대비 오차 · 호출 수 · 시간 · seed 간 편차 · 달성률
- [ ] `multi_objective.py`: NSGA-II, 무릎점, 단일목적 해와의 일관성
- [ ] 원료 고정 시 Ni–Co 예측 표면 상충 여부 그림

6단계 — 검증 · 시각화 · 문서

- [ ] 등고선(황산 × 온도, 시간 고정) + 95% 경계 + 실행 가능 영역 + 추천점 + 학습 데이터 점
- [ ] 민감도: 목표 94 → 97 변화에 따른 추천 황산량
- [ ] 경향성: 불순물 많을수록 더 많은 황산 · 높은 온도를 추천하는가
- [ ] 등급별 목표 달성 가능 배치 비율
- [ ] 백테스트: **out-of-fold 예측**으로 목표 달성 6건 원료에 추천 → 실제 레시피와 비교
- [ ] 보고서 Module 3 섹션 초안 (수치 자리만 비움)

---

7. 검증 기준

검증	통과 기준
제약 위반	모든 추천에서 0건

검증	통과 기준
반올림 재검증 (Optuna 결과)	재예측 후에도 목표 만족 100%
정답 검증 (해석 모델)	Grid 결과가 이분 탐색 정답과 격자 간격 이내
알고리즘 일관성	Grid · Optuna · GA 최적 비용 차이 작음
파레토 일관성	마진 = 0 지점이 단일목적 해와 일치
달성 불가	Ni ≥ 120 같은 목표에서 `infeasible`
범위 밖 원료	극단 성분에서 `out_of_domain`
결정성	같은 seed → 같은 결과

8. 결정 기록

날짜	결정	이유	상태
2026-10-10	결정 변수 3개 (황산 · 온도 · 시간)	명세 문장 간 불일치를 모두 충족	멘토 확인 전
2026-10-10	`feed_kg`는 주어진 값	설비 용량으로 정해지는 배치 크기	멘토 확인 전
2026-10-10	목표 제약 + 황산 최소화, 달성 불가 시 사전식	시나리오 부합, λ 제거	확정
2026-10-10	z = 1.28 (판정) / 1.645 (표시) 분리	단측 90%와 양측 90% 구간은 다른 양	확정
2026-10-10	Grid를 현장 단위로 생성	9.3만 점도 빠르고 반올림 단계 제거	확정
2026-10-10	다목적 = (안전 마진, 황산량)	Ni–Co r = 0.915	예측 표면 확인 후 확정
2026-10-10	원료 입력 결측은 거절	대치는 P1 영역, 추천 근거 왜곡 방지	팀 합의 전

9. 실행 방법

구현 후 채운다. 예정 명령:

```
python -m module3_optimization.recommend --feed feed.json --algo grid
python -m module3_optimization.recommend --batch-id BP-2025-0412
python -m module3_optimization.recommend --all --out results.csv
python -m module3_optimization.compare
pytest module3_optimization/tests
```